Protocolo Experimental

Implementação de Camada de Rede Assíncrona em Android

Grupo 9| Unidade Curricular de PDM

1. Enquadramento Teórico

A comunicação entre aplicações móveis (clientes) e servidores externos assenta no protocolo HTTP, utilizando o modelo Cliente-Servidor. Nesta arquitetura, a aplicação Android efetua pedidos ao servidor (API remota), que processa a informação e devolve uma resposta. Para garantir uma comunicação padronizada, utilizam-se os princípios REST, onde ações específicas são representadas por verbos HTTP. Neste projeto, destaca-se o uso do verbo **GET** para a leitura e obtenção de dados do servidor.

O formato de eleição para a transferência de dados em APIs modernas é o **JSON** (JavaScript Object Notation), um formato de texto leve, estruturado em pares de chave-valor. Para que a aplicação em Java consiga interpretar esta resposta de forma nativa, ocorre um processo de **desserialização**, que consiste em mapear as chaves do texto JSON para variáveis privadas de objetos no código (POJOs), frequentemente recorrendo a *getters* para aceder à informação em segurança.

O processamento desta comunicação no ecossistema Android exige bibliotecas especializadas. O **Retrofit** atua como uma biblioteca de alto nível, convertendo os *endpoints* da API em interfaces Java simples. Por baixo deste, atua o **OkHttp**, um cliente de rede de baixo nível responsável por abrir a ligação real à internet, gerir tempos de espera (*timeouts*) e tratar erros de conexão. Em paralelo, o **Gson** é incorporado para automatizar totalmente o processo de desserialização.

1.1. Assincronismo e Gestão de Threads

O sistema Android impõe regras rígidas de execução para garantir a fluidez da interface. A *Main Thread* (ou UI Thread) é exclusivamente responsável por desenhar o ecrã e registar interações (toques). Caso uma operação de rede, que pode demorar vários segundos, seja executada nesta linha principal, o sistema bloqueia a interface e lança uma exceção de ANR (*Application Not Responding*).

Para resolver este problema de assincronismo num projeto desenvolvido em Java, utiliza-se a classe ExecutorService. Esta ferramenta delega o pedido de rede para uma *Worker Thread* secundária, mantendo a *Main Thread* livre. Quando os dados são finalmente recebidos e processados, a comunicação de volta para a interface gráfica é feita de forma segura através de LiveData. Utilizando o método postValue(), o resultado é agendado e entregue na *Main Thread* de forma assíncrona, respeitando o ciclo de vida da aplicação e evitando fugas de memória (*memory leaks*) caso o ecrã seja fechado antes da resposta chegar.

2. Objetivos

- Implementar a camada de rede numa aplicação Android utilizando a linguagem Java e os princípios REST.
- Configurar o Retrofit e o OkHttp para efetuar pedidos à API *justmeme.wtf* sem bloquear a *Main Thread*.
- Desserializar as respostas JSON automaticamente para modelos nativos utilizando a biblioteca Gson.
- Gerir estados de interface (*Loading*, *Success*, *Error*) através de LiveData e padrões arquiteturais.
- Implementar tolerância a falhas com a captura de exceções de rede e a criação de uma *cache* local (SharedPreferences) para disponibilizar o conteúdo em modo *offline*.

3. Configuração Utilizada

- **Ambiente e Linguagem:** Android Studio (Projeto em Java).
- **Bibliotecas de Rede:** Retrofit2, OkHttp3 e GsonConverterFactory.
- **Motor de Assincronismo e Estado:** ExecutorService (Java) e MutableLiveData (Android Architecture Components).
- **Armazenamento Local:** SharedPreferences integradas com Gson para *cache* de dados.
- **Emulador:** Dispositivo virtual Android com permissões de acesso à internet ativadas no AndroidManifest.xml.

4. Dados de Entrada e Parâmetros

O projeto comunica com o URL Base https://justmeme.wtf/. O *endpoint* principal alvo do teste é /api/v1/templates, requisitado através de um verbo GET. Espera-se como resposta um objeto JSON que contém uma estrutura de lista (*array*) na qual cada elemento representa um meme, caracterizado pelas chaves "id", "name" e "url".

5. Procedimento Experimental

1. **Definição dos Modelos de Dados:** Criação da classe Java MemeTemplate. Definição dos atributos privados e utilização da anotação @SerializedName para assegurar o mapeamento exato das chaves do JSON, independentemente de eventuais ofuscações de código.
2. **Configuração do Cliente de Rede:** Instanciação do OkHttpClient com tempos de espera (*connectTimeout* e *readTimeout*) reduzidos para **10 segundos**, simulando condições de falha mais rápidas para a demonstração. A seguir, construção do objeto Retrofit integrando o *URL Base*, o cliente OkHttp e o *ConverterFactory* do Gson.

3. **Execução Assíncrona:** Invocação do *endpoint* na camada lógica (ViewModel) com recurso ao ExecutorService, deslocando o processo bloqueante execute() para uma *Worker Thread*.
4. **Tratamento de Exceções:** Implementação de blocos try/catch na chamada de rede para intercetar falhas como SocketTimeoutException (demasiado tempo de espera) e IOException (falta de ligação).
5. **Atualização da UI:** Injeção dos dados obtidos no MutableLiveData através do método postValue(), alterando o estado global da aplicação entre *Loading* (exibindo a barra de progresso), *Success* (desenhando a lista) e *Error* (mostrando opções de recuperação).
6. **Gravação em Cache:** Em caso de sucesso, serialização imediata da lista para JSON e gravação em SharedPreferences. Em caso de falha de rede, leitura deste espaço de memória para apresentar os últimos dados guardados.

6. Métricas e Resultados Esperados

- **Sucesso de Comunicação:** O servidor devolve um código de estado HTTP **200 OK**. A resposta é traduzida e desenhada na interface, mantendo a *Main Thread* completamente fluida a 60 *frames* por segundo.
- **Tratamento de Timeout:** Caso a API demore mais de 10 segundos, o OkHttp deve abortar a ligação. A aplicação não deve falhar (*crash*), mas sim transitar para o estado *Error* e apresentar um ecrã com o botão "Tentar novamente".
- **Resiliência Offline:** Numa execução sem ligação à internet, a aplicação deve apresentar sucesso na leitura das SharedPreferences, indicando na *interface* que os dados mostrados provêm do último acesso.